

COMP1531

Software Engineering Fundamentals

Week 3

Correctness - Dynamic Verification

In This Lecture

— — —

Why?

- Writing tests are critical to **ensure an application works**
 - behaves as expected and meets requirements
- Approaching testing the right way will **yield better results**
 - Saves time, cost and major problems
- We need to understand the characteristics of a programming language, to better understand how to **prevent bug**

In This Lecture

— — —

What?

- Abstraction & Black boxes
- Design by contract
- Vitest
- Safety

Quality control when developing software is important.

In extreme cases, a bug or defect can degrade interconnected systems or cause serious issues.

Software Testing is Important

— — —

- Nissan having to recall over 1 million cars due to a software defect in the airbag sensor detectors.
- Software bug caused the failure of a USD 1.2 billion military satellite launch.
- 2024 Cloudstrike outage could have been prevented by testing.
- Software failures cost the US economy \$1.1 trillion in 2016
 - And impacted 4.4 billion customers.

Software Testing is Important

— — —

- Software testing shows systems are operating as expected
 - ... sort of



Testing In The SDLC

— — —



[source](#)

Testing In The SDLC

— — —

- Software testing is the **process of evaluating and verifying** that a software product or application **does what it is supposed to do.**
- The benefits of testing include:
 - preventing bugs,
 - reducing development costs and
 - improving performance.
- Because many teams immediately test the code they write, the testing phase often runs parallel to the development phase.

Software Testing

— — —

There are many different types of software tests:

- Acceptance testing
- Integration testing
- Unit testing
- Usability testing
- Functional testing
- Performance testing
- ...

Software Testing

— — —

There are many different types of software tests:

- Acceptance testing
 - Meets the business requirements?
- Integration testing
 - Working well together?
- Unit testing
 - Individual methods?
- Usability testing
 - Good to use?
- Functional testing
 - Verify just the output?
- Performance testing
 - Speed?
- ...

Software Testing

- **Dynamic verification**
 - exercising and observing product behaviour
- **Why is it dynamic?**
 - The system is executed with test data and its **operational behaviour** is observed.
 - “run-time-checks”
- **Static Verification**
 - Type safety/correctness
 - Control flows
 - Typescript
- **Why is it static?**
 - checking properties of code **without running it**
 - “compile-time-checks”

Software Safety

— — —

- The goal of testing is to make software safe
 - This is different than making software secure
- **Safety**
 - Prevention from accidental misuse
- **Security**
 - Prevention from deliberate misuse

Software Safety

- Software becomes unsafe when **unexpected or unintended behaviours are allowed to happen**
 - Reading uninitialised memory
 - Writing outside array bounds

Thankfully we're using a normal and sensible language that would never let us do anything bad or weird!



FunFact

— — —

- 94% of spreadsheets have an error in them
 - Because people do not verify their correctness

Example: Memory Safety

— — —

- C is **not considered** a memory safe language. However, Javascript is.
- Javascript prevents access to memory that hasn't been initialised or allocated. It does this by dynamically checking at runtime.
- Whereas in C there is **no bounds checking** performed for array accesses. Pointers can be dereferenced even if they don't point to allocated memory.
- C prioritises performance over memory safety. Javascript vice-versa.

Static Verification

— — —

- Static verification is usually considered a more robust and reliable form of testing.
- However, it's limited as not everything can be verified statically.
- In this course we will talk about **Static Typing** and **Linting**. We will talk about static verification later.

Dynamic Verification

— — —

Dynamic testing is about making your program more robust in the face of real and inevitable things that go wrong while a program is running.

Typically falls into one of two categories:

- **Testing in the small**
- **Testing in the large**

“Testing Shows The Presence, Not The Absence Of Bugs” — Edsger

W. Dijkstra

Dynamic: Testing In The Small

— — —

- Unit testing is the process of testing individual components in isolation.
- Unit tests are often just testing particular functions in isolation.

Dynamic: Testing in The Large

— — —

- Exposes defects in integrated systems
 - Buttons that call functions
 - Functions that call other functions
- Module tests
 - Testing entire module
- Integration tests
 - Testing modules working together
- Systems tests
 - Testing entire system

How to Test?

How would you test this bit of code?

```
function add_evens(limit) {  
  //Adds even numbers from 1 to the limit  
  numbers = [];  
  for (let i = 0; i < limit; i++) {  
    if (i%2 == 0) {  
      numbers.push(i);  
    }  
  }  
  return numbers;  
}
```

Would You Do This?

— — —

```
function add_evens(limit) {  
    //Adds even numbers from 1 to the limit  
    numbers = [];  
    for (let i = 0; i < limit; i++) {  
        if (i%2 == 0) {  
            numbers.push(i);  
        }  
    }  
    return numbers;  
}  
  
console.log(add_evens(12));  
console.log(add_evens(7));  
console.log(add_evens(-2));  
console.log(add_evens(15));
```

However...

— — —

- That is not testing
 - That is debugging
- It's only testing if it scales well
 - We can't `console.log` forever

Black-Box Testing

— — —

- We focus on abstraction
 - We don't worry about the underlying detail
- Think about driving a car, or using your PC
 - Do we care how the car works? Or just that it works?
- This is called **black-box testing**

Blackbox Testing, Abstraction

— — —

- When we're testing our code, we always want to view the functions we're testing as abstractions / black boxes.
- Can we write tests for these stub functions ?

```
1 // Returns a new string with vowels removed
2 function removeVowels(string) {
3   return 0;
4 }
5
6 // Calculates the factorial of a number
7 function factorial(num) {
8   return 0;
9 }
```

blackbox.js

Blackbox Testing, Abstraction

— — —

- We can
 - We don't need to know how the function is implemented to test the function
 - Implementation itself doesn't actually impact what we want to happen

```
1 // Returns a new string with vowels removed
2 function removeVowels(string) {
3   return 0;
4 }
5
6 // Calculates the factorial of a number
7 function factorial(num) {
8   return 0;
9 }
10
11 console.log(removeVowels('abcde') === 'bcd');
12 console.log(removeVowels('frog') === 'frg');
13 console.log(factorial(3) === 6);
14 console.log(factorial(5) === 120);
```

blackbox2.js

Vitest : Testing JS

- To test at scale we need a real testing [framework](#). [Vitest](#) is a [popular](#) framework for nodejs/javascript. It is installed with NPM.
- Let's install **Vitest**:
 - **npm install --save-dev vitest**
- We add **--save-dev** because its a dependency being used for development and testing, but wouldn't be used in production.

Vitest : Testing JS

Let's setup a simple example and write some Vitest tests for our function.

```
1 // Returns a new string with vowels removed
2 function removeVowels(string) {
3   string = string.replaceAll('a', '');
4   string = string.replaceAll('e', '');
5   string = string.replaceAll('i', '');
6   string = string.replaceAll('o', '');
7   string = string.replaceAll('u', '');
8   return string;
9 }
10
11 export { removeVowels };
```

[removeVowels.js](#)

Vitest: Testing JS

— — —

```
1  import { describe, test, expect } from 'vitest';
2  import { removeVowels } from './removeVowels.js';
3
4  describe('removeVowels', () => {
5    test('deals with no vowels', () => {
6      const example1 = removeVowels('bcd');
7      const example2 = removeVowels('lkj');
8      expect(example1).toEqual('bcd');
9      expect(example2).toEqual('lkj');
10   });
```

Now we can run

`./node_modules/.bin/vitest removeVowels.test.js`

[Using Matchers in Vitest](#)

Vitest: Testing JS

We can add **"test": "vitest run"** to our *package.json* scripts.

Now when we run **npm run test**
it will execute :

`./node_modules/.bin/vitest.`

```
1  {
2    "name": "3.1-dynamicverification",
3    "version": "1.0.0",
4    "description": "Lecture Demo",
5    "main": "index.js",
6    "type": "module",
7    "scripts": {
8      "test": "vitest run"
9    },
10   "author": "Shaveen Singh",
11   "license": "ISC",
```

Let's Add More Tests!!

— — —

```
1  import { describe, test, expect } from 'vitest';
2  import { removeVowels } from './removeVowels.js';
3
4  describe('removeVowels', () => {
5    test('deals with no vowels', () => {
6      const example1 = removeVowels('bcd');
7      const example2 = removeVowels('lkj');
8      expect(example1).toEqual('bcd');
9      expect(example2).toEqual('lkj');
10   });
11   test('deals with only vowels', () => {
12     expect(removeVowels('aei')).toEqual('');
13     expect(removeVowels('oiu')).toEqual('');
14   });
15   test('deals with starting vowels', () => {
16     expect(removeVowels('ant')).toEqual('nt');
17     expect(removeVowels('old')).toEqual('old');
18   });
19   test('deals with ending vowels', () => {
20     expect(removeVowels('bee')).toEqual('b');
21     expect(removeVowels('hi')).toEqual('h');
22   });
23   test('deals with complex words', () => {
24     expect(removeVowels('cannot')).toEqual('cnnt');
25     expect(removeVowels('delicious')).toEqual('dlcs');
26   });
27 });
```

Vitest: Testing JS

— — —

- The general structure is that you have a series of **outermost describes** that are for abroad area (e.g. auth).
- Then some describes inside that might cover for **instance a particular function**.
- Then inside that, we have **tests for each property** or **use case** we're looking to test.
- Then inside that we use one of **Vitest's expect functions** ([please see docs](#))

Design By Contract

- When we're testing or implementing a function, we will typically be working with information that tells us the constraints placed on at least the inputs.
- The documentation can come in a variety of forms.
- This information tells us what we do and don't need to worry about when writing tests.

```
1 // Returns a new string with vowels removed
2 // Input is a non-empty string type
3 // Return type is another string
4 function removeVowels(string) {
5   return 0;
6 }
7
8 // Calculates the factorial of a number
9 // Input is a number between 1 and 10
10 // Output is a positive number
11 function factorial(num) {
12   return 0;
13 }
```

design_contract.js

Behaviour

— — —

- When we look at how programs respond to being used, we often use the word **behaviour** to describe what they do under certain conditions.
- When we test, **we are essentially just testing the behaviour** of a program.
- However, it's not always possible to understand the behaviour of every possible scenario from a spec perspective, often either due to:
 - It is not feasible to define every possible edge case
 - The implementation details are not critical to the spec

Undefined Behaviour

— — —

- Undefined behaviour is where a behaviour is not defined at all, often because the use case is beyond a reasonable scope of expectation, and that behaviour generally shouldn't occur within a reasonable usage of the system.
- An example might be trying to find the square root of negative 1.

Implementation-Defined Behaviour

— — —

Implementation defined behaviour is where a behaviour needs to be defined and considered, but it is up to the individual implementing it to make decisions about it.

An example might be whether text inputs are ASCII or UTF-8 encoding.

Feedback

— — —



Or go to the [form here](#)

COMP1531

Software Engineering Fundamentals

Correctness - Code Coverage

In this Lecture:

— — —

- We need a mechanism to tell how much our tests actually test the code
“Out of all the code I wrote, how much did my tests actually check?”
- What?
 - Coverage
 - Code Coverage
 - Coverage in Vitest



Coverage

— — —

- A general measure of how thorough our tests are.
 - *how much* of your program's code is actually executed (or “covered”) when you run your tests.

Test Coverage: a measure of how much of the feature set is covered with tests. This is often left to human judgement.

Code coverage: a measure of how much code is executed during testing. This can be computed and quantified using scripts.



Code Coverage

— — —

- is a metric - it measures how much of your code has been executed during testing.

Eg:

File: books_server.js

Lines: 75% (30/40 lines covered)

What other metrics can we use for code coverage?



Coverage in Vitest

Vitest (the testing framework used in JavaScript/TypeScript) also has **code coverage** support.

To check for test coverage, we will also install [@vitest/coverage-v8](https://www.npmjs.com/package/@vitest/coverage-v8).

```
npm install --save-dev @vitest/coverage-v8
```

Example Vitest Coverage Output:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	88.88	87.5	100	88.88	
grade.js	88.88	87.5	100	88.88	13



Coverage in Vitest

— — —

Vitest will:

- Run all your tests
- Record which lines, functions, and branches of code were executed
- Produce a detailed coverage report in the console and usually in :
`coverage/index.html`

Open using terminal command: `open coverage/index.html`



Let's Check Some Code Coverage!

-- --

We've got two pieces of sample code. Let's write tests with Vitest and the `--coverage` flag (in package.json) and use coverage to help us write better tests!

Example: computeGrade

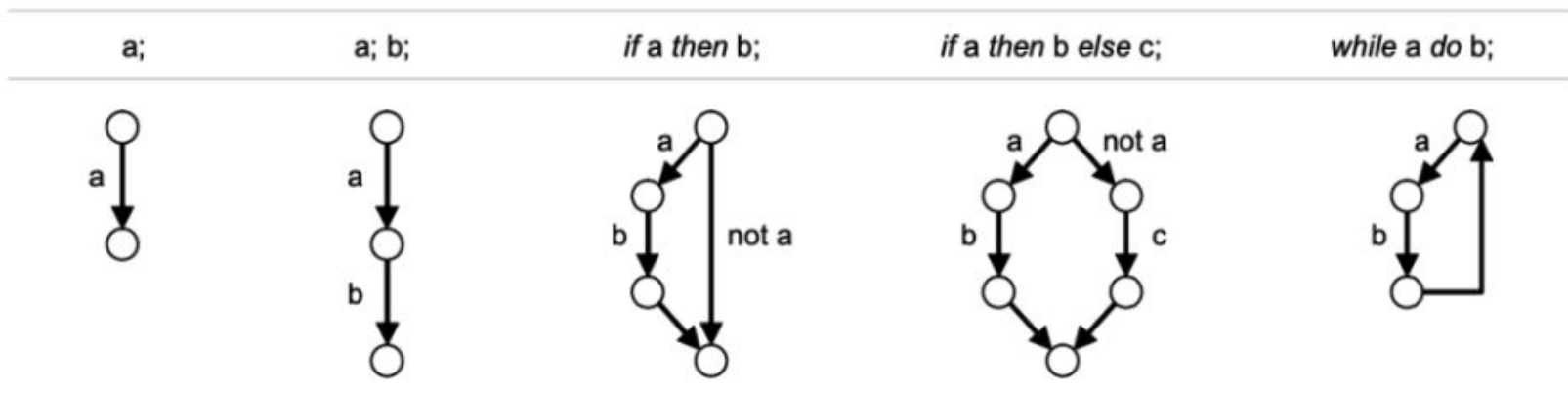
— — —

```
function computeGrade(mark) {  
  if (mark >= 85) {  
    return "HD"; // High Distinction  
  } else if (mark >= 75) {  
    return "DN"; // Distinction  
  } else if (mark >= 65) {  
    return "CR"; // Credit  
  } else if (mark >= 50) {  
    return "PS"; // Pass  
  } else {  
    return "FL"; // Fail  
  }  
}
```

computeGrade.js

Branch Coverage Checking

- For lines that can potentially jump to more than one other line (e.g. if statements), check how many of the possible branches were taken during execution
- Done automatically with Vitest.
- Sometimes referred to as edge coverage.





Common Coverage Metrics in Vitest

-- -- --

Metric	Meaning
Statements	Percentage of all code statements executed
Branches	If/else or switch-case paths covered
Functions	How many functions were called
Lines	Actual source code lines covered

Summary

— — —

- Code coverage is useful.
- It's more important to look at what's not covered than the coverage percentage.
- Branch coverage is a more accurate measurement so you should use it instead of statement coverage.
- Like all measurements, it's important to understand what meaning to attach to it.
- Wanna know more about different types of code coverage? Take COMP3142!

Feedback

COMP1531 Feedback Survey



gradeFromScore(score)

```
┌── Yes → 'FL'   (score ≤ 49)
score ≤ 49?
└── No
    ┌── Yes → 'PS'   (50–64)
    score ≤ 64?
    └── No
        ┌── Yes → 'CR'   (65–74)
        score ≤ 74?
        └── No
            ┌── Yes → 'DN'   (75–84)
            score ≤ 84?
            └── No → 'HD'   (85–100)
```